

通知服务系统设计：一次完整的推演记录

题源： Coupang 面经（1point3acres）中的原题，在店面与 onsite 两个不同轮次都出现过，是该公司频率最高的系统设计题。

原始题面： 设计一个通知服务。内部用户可以配置规则，系统根据规则向不同类型的外部用户发送不同类型的通知。要求高并发、低延迟。

本文记录的是**推演过程本身**，包括中途被数字否定的几个方案，以及每次否定后架构如何收敛。

1. 澄清与边界

题面故意留白，开场必须问清三件事。这不是形式——**SLO 的取值直接决定架构分叉**。

1.1 必问的 SLO

问题	为什么决定架构
端到端延迟要求？	秒级 → 纯轮询即可；毫秒级 → 必须有长连接推送通道
单条通知的最大扇出？	扇出上限决定能否逐用户物化
读写比？	决定优化重心
投递语义？	at-most-once / at-least-once，决定是否需要重试与幂等

1.2 两层规则的区分

题面中的”规则”是歧义点，必须拆成两层，否则会答错范围：

- **产生规则：** 什么事件该给谁发什么通知。例如某 ML 模型输出超过阈值、订单状态变更。这一层属于上游业务系统与配置层。
- **分发规则：** 通知产生之后怎么发、发几次、频控、去重。这一层是本服务的职责。

在面试中把这个区分说出口本身就是加分项，因为它说明候选人理解了系统边界。

1.3 本文的边界声明

上游各业务系统产生 trigger 事件；内部用户配置规则，把 trigger 映射成（receiver 集合，通知类型）。本设计从这个映射的产物开始，重点在下游的分发链路。

边界必须明说。 脑子里有边界但不说出口，面试官只能判断为”漏了”。

2. 数据流

1. 上游业务系统产生 trigger 事件 → 规则求值
2. 物化成通知记录： receiver_id, notification_type, metadata, timestamp，按时间排序
3. 事件驱动：下游消费服务读取新产生的通知
4. 按用户去重
5. 用户在线 → 推送信号；用户离线 → 落库等待拉取
6. 客户端上线或定期轮询时拉取

产品形态换系统成本： 小铃铛只需展示最近十几条，客户端可缓存旧消息。这一个产品决策就能大幅降低读写压力——在系统设计中主动提出产品侧的取舍，是高级信号。

3. Push 还是 Pull：被 SLO 分开的两条路

3.1 分叉点

这是整道题的第一个关键判断：**端到端延迟 SLO 决定架构。**

- SLO 宽松（秒级可接受）→ 客户端轮询。实现简单，无需维护连接
- SLO 严格（毫秒级）→ 必须服务端推送。需要长连接、在线状态维护、扇出编排

两条路的复杂度差一个数量级。在面试中主动指出这个分叉、并向面试官索要 SLO，比直接选一条路更好。

3.2 为什么纯 Pull 不够

轮询对信息流（newsfeed）足够，但对通知不够：订单支付确认、紧急告警这类要求秒级甚至更快。**几秒一次的轮询意味着平均延迟为轮询周期的一半**，对实时通知不可接受。

3.3 为什么纯 Push 也不行

推送的成本在**连接**，不在数据。一条扇出百万的通知，如果逐个连接推送：

- 服务端需维护百万级并发长连接
- 瞬时带宽与连接管理开销极高
- 大量目标用户并不在线，推送直接浪费

参照物：Amber Alert 之所以能全量推送，是因为它走运营商级的 cell broadcast（基站向覆盖区内所有设备广播），**不经过应用服务器**。普通应用没有这个能力。

3.4 最终方案：信号 + 拉取

统一通道，不分大小扇出：

1. 服务端推送的只是**信号**（“你有新消息”），载荷几十字节
2. 客户端收到信号后主动拉取内容
3. 客户端同时保留定期兜底轮询（如 10 分钟一次），漏推可被自动补齐
4. 客户端按 `notification_id` 去重

这样 push 只负责降低延迟，不承担可靠性。 推送失败不重试——大不了等下一次兜底轮询。这个取舍让整个推送链路可以做得非常简单。

4. 长连接与在线状态

4.1 组成

- **连接网关层**：每台维护数十万条长连接
- **注册中心**：`user_id → gateway_id`，内存 KV，带 TTL
- **心跳续期**：客户端定期心跳，超时未续即视为离线

4.2 心跳频率的取舍

心跳频率取决于能容忍多长的“假在线”窗口——用户已掉线但服务端仍认为在线，这期间的推送会丢。

频率	假在线窗口	写压力（200 万在线）
5 秒	~15 秒	40 万 QPS
30 秒	~90 秒	6.7 万 QPS
60 秒	~180 秒	3.3 万 QPS

选 30 秒心跳、90 秒 TTL。理由：

- 移动端每次心跳唤醒射频，5 秒一次对电池不友好，且 iOS/Android 会在后台限制该频率
- 假在线的代价极低——推送丢了有兜底轮询，不是数据丢失
- 写压力差 6 倍

网关本地聚合：客户端心跳先落在网关内存，网关按更长周期（如 30 秒）批量向注册中心续期。用户侧仍是密集心跳（掉线检测快），注册中心的写压力降低一个数量级。

5. 规模估算

5.1 假设

项	值
注册用户	10 亿
DAU	1000 万
峰值同时在线	200 万
通知产生峰值	5 万条/秒
每用户保留	最近 1000 条
单条通知	时间戳 8 字节 + 内容 100 字节 ≈ 108 字节，压缩后 22 字节
机器数	200 台

5.2 存储

每用户：1000 条 × 22 字节	= 22 KB
全量：10 亿 × 22 KB	= 22 TB
单机：22 TB ÷ 200	= 110 GB
三副本：66 TB，单机 330 GB	

存储不是瓶颈。 22 TB 分摊到 200 台完全可行，写入 QPS 才是约束。

注意：存储必须按**全量用户**估算，与在线与否无关——通知不管用户在不在线都要落库。DAU 只用于估算读 QPS。

5.3 分片

- 分片键：user_id hash
- 每台约 **500 万用户**
- 同一用户的通知落在同一台，读取是单分片查询
- 淘汰：每用户 1000 条上限，写入时滚动淘汰，保证总量不增长

5.4 三档扇出分级

这是本题最关键的设计决策。先看一个反证：

若 5 万 QPS 的通知每条扇出 100 万，逐用户物化就是 每秒 500 亿次写入 ，按 22 字节算是 1.1 TB/s 。200 台每台需写 5.5 GB/s——超过单机 NVMe SSD 的顺序写上限（1~3 GB/s）。加到 2000 台每台仍需 5000 万条/秒。
结论：这个负载不可能逐用户物化，任何机器数都救不了。

因此必须按扇出分档：

档	量	扇出	路径	存储写入
常规	4 万条/秒	≤ 100	写扩散：按 user_id hash 路由到分片，追加进收件箱；在线用户额外推送信号	峰值 400 万/秒
紧急广播	1 万条/秒	到 region	不物化。连接层按地理分组扇出，客户端连接时注册 region:xxx	~1 万/秒（仅落一条记录）
非紧急大扇出	剩余	大	不物化。落一条 campaign 记录 + 受众条件，客户端拉取时读侧合并	条数级，可忽略

分流判据： 通知入口携带 priority 与 estimated_fanout 两个属性，据此路由。扇出阈值按写入预算倒推： $200 \text{ 台} \times \text{单机 } 2 \text{ 万 QPS} \div 4 \text{ 万条/秒} \approx 100$ 。

核心取舍： 写入预算全部留给第一档；后两档用”不 fanout” 换掉写放大，代价挪到读侧合并与连接层扇出。

5.5 为什么批处理管道也救不了大扇出

一个自然的想法是攒批跑 shuffle（MapReduce / Presto / Spark），把 notif → user 转成 user → notif 再批量导入。算一下：

攒 1 分钟：5000 条/秒 × 60 秒 × 10 万扇出 = 300 亿条中间记录

按 22 字节：660 GB 的 shuffle 中间数据

写入端：200 台每台吸收 1.5 亿条 / 3.3 GB ≈ 55 MB/s — 批量导入可行

写入侧勉强能扛，但**存储增长率**成了新瓶颈：每分钟新增 660 GB，一小时 40 TB，半小时就撑爆 22 TB 的总容量。

批处理改变的是写入路径的效率，不改变数据总量。 大扇出通知的正解只能是不物化。

5.6 单机资源核算

每台机器上放三样东西：

数据	分布方式	单机大小
user notif table	user_id hash 分片	110 GB（500 万用户 × 22 KB）
region notif table	全量复制	~300 MB（10 万区域 × 100 条 × 30 字节）
user_id → gateway	user_id hash 分片	1 MB（200 万在线 ÷ 200 = 1 万条 × 100 字节）

region table 全量复制的好处：客户端一次拉取的三个子请求（个人通知、区域通知、大扇出合并）**全部落在同一台机器**，无需跨机查询。

5.7 写入与推送 QPS

写入：

峰值 400 万次/秒 ÷ 200 台 = 每台 2 万 QPS（追加写）

每次写入 22 字节 → 每台 440 KB/s — 网络与磁盘均可接受

推送信号：

该机在线用户占比 = $(200 \text{ 万} \div 200) \div 500 \text{ 万} = 0.2\%$

每台推送 QPS = $2 \text{ 万} \times 0.2\% = 40 \text{ QPS}$

全局 = $40 \times 200 = 8000 \text{ QPS}$

假设通知接收者在用户中均匀分布。真实系统中活跃用户收到的通知更多，该比例会偏高，但作为估算量级正确。

读取（拉取）：

```
200 万在线 ÷ 10 秒轮询周期 = 20 万 QPS
每台 = 20 万 ÷ 200 = 1000 QPS
```

一次拉取在本机完成三份数据的查询与合并。

出口带宽：

```
20 万 QPS × 平均 1 KB = 200 MB/s
每台 1 MB/s — 可忽略
```

结论：读侧压力（20 万 QPS）是推送侧（8000 QPS）的 25 倍。这印证了架构判断：推送只是降低延迟的手段，拉取才是主通道。

6. 读侧合并

客户端一次拉取需要合并三个来源：

- 1. 个人收件箱 —— 本机按 user_id 直接查询
- 2. 区域广播 —— 本机 region table，按用户所属区域 + last_read_ts 取增量
- 3. 大扇出内容 —— 按受众条件判定命中，读时合并

三者按时间归并，靠 notification_id 去重。

离线补齐：用户表记录 last_active_time。用户上线时按该时间戳拉取增量，并施加”最多 N 条”的截断，避免长期离线用户上线时拉取过多历史。

受众判定：定向广播（某城市、某 segment）的受众存为条件表达式而非展开的用户列表，拉取时按用户属性求值。这是避免写放大的关键——存条件的代价是 O(1)，展开列表的代价是 O(受众规模)。

7. 工业界对照

场景	工业界做法	关键点
大扇出推送	topic 订阅。客户端订阅 topic，服务端向 topic 发一条，推送基础设施做扇出（FCM/APNs 原生支持）	扇出发生在连接层，不在存储层
Feed 分发	混合 fanout。普通用户写扩散，大 V 读扩散，阈值约几万粉丝（Twitter 的经典做法）	按扇出规模分流
营销通知	存条件不存列表（Braze、Airship 一类）。发送时才对用户库跑批量扫描	天然异步、分钟级，不承诺秒级送达

映射回本题的三档：实时小扇出 → 写扩散 + 推送信号；实时大扇出 → topic/区域广播，连接层扇出；非实时大扇出 → 存条件，异步批量。

分档的维度是「实时性 × 扇出规模」的组合，不是单一维度。

8. 失败模式

失败	处理
通知表分片不可用	副本切换。切换窗口内的写入按 <code>priority</code> 分流：高优先级进重试队列，低优先级直接丢弃（用户漏看一条营销消息不致命，订单状态不能丢）
网关整机宕机	数十万连接同时断开，客户端重连到其他网关并重新注册。注册中心的 TTL 保证陈旧记录自动过期
注册中心不可用	推送降级为不可用，全部退化为轮询。 这是系统能优雅降级的关键——因为推送本就不承担可靠性
推送失败 / 漏投	不重试。依赖客户端 10 分钟兜底轮询补齐
上游 trigger 重复投递	消息队列通常是 at-least-once。以 <code>(trigger_id, user_id)</code> 为幂等键去重，否则用户会看到重复通知
消费积压	背压 + 降级：按 <code>priority</code> 丢弃低优先级、延迟处理非实时档。 队列深度是最早的告警信号
数据丢失	写前日志 + 多副本。系统按峰值 QPS 设计，留有余量

9. 监控指标

业务层 - 送达率、点击率 - 按 `notification_type` 分的转化率

系统层 - 端到端延迟 P50 / P99（从 trigger 产生到用户设备收到） - 推送成功率 - **拉取空返回率** —— 若 99% 的轮询都是空返回，说明轮询间隔可以拉长，直接节省成本 - 队列深度（积压的最早信号） - 各分片写入 QPS 的倾斜度

容量层 - 在线连接数 - 注册中心命中率（假在线比例） - 存储增长率

10. 设计要点回顾

- SLO 决定架构分叉。** 秒级用轮询，毫秒级必须有推送。这个问题要在设计开始前问出口。
- 两层规则必须区分。** 产生规则（谁收到什么）与分发规则（怎么发、频控）属于不同层次；把边界明说出来。
- 推送不承担可靠性。** 推送只负责降低延迟，失败不重试，由客户端兜底轮询保证最终送达。这个取舍让整条推送链路可以做得很简单，也让注册中心故障时能优雅降级。
- 扇出规模决定物化策略。** 小扇出写扩散，大扇出读扩散。判据是写入预算，不是通知类型。
- 数字会否定方案。** 5 万 QPS × 百万扇出 = 500 亿次写/秒，任何机器数都不可行——这个计算直接淘汰了逐用户物化。批处理管道同样救不了，因为它改变的是写入效率而非数据总量。**在系统设计中，先算数再定方案。**
- 存储按全量估，读 QPS 按 DAU 估。** 两者的分母不同，混用会得出错误结论。
- 读侧压力远大于推送侧。** 本设计中是 25 倍。这决定了优化重心应放在拉取路径而非推送路径。